In [1]:
```
import os
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms, models
from torch.optim.lr_scheduler import StepLR
from tqdm import tqdm
DATA_DIR = "data"
BATCH_SIZE = 32
NUM_CLASSES = 2
EPOCHS = 10
LR = 1e-4
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
train_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
                         [0.229, 0.224, 0.225])
])

val_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
                         [0.229, 0.224, 0.225])
])

train_dataset = datasets.ImageFolder(os.path.join(DATA_DIR, "train"), transfor
val_dataset   = datasets.ImageFolder(os.path.join(DATA_DIR, "val"), transform=

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True,
val_loader   = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False, r


model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V1)

# Freeze all layers first
for param in model.parameters():
    param.requires_grad = False


for param in model.layer4.parameters():
    param.requires_grad = True


num_features = model.fc.in_features
model.fc = nn.Sequential(
    nn.Linear(num_features, 512),
    nn.BatchNorm1d(512),
    nn.ReLU(),
    nn.Dropout(0.5),
    nn.Linear(512, NUM_CLASSES)
)

model = model.to(DEVICE)


criterion = nn.CrossEntropyLoss()
```

```
optimizer = optim.Adam(filter(lambda p: p.requires_grad, model.parameters()),
scheduler = StepLR(optimizer, step_size=5, gamma=0.1)


def train_one_epoch(model, loader):
    model.train()
    running_loss = 0
    correct = 0

    for images, labels in tqdm(loader):
        images, labels = images.to(DEVICE), labels.to(DEVICE)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        _, preds = torch.max(outputs, 1)
        correct += torch.sum(preds == labels)

    epoch_loss = running_loss / len(loader)
    epoch_acc = correct.double() / len(loader.dataset)

    return epoch_loss, epoch_acc.item()


def validate(model, loader):
    model.eval()
    running_loss = 0
    correct = 0

    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(DEVICE), labels.to(DEVICE)

            outputs = model(images)
            loss = criterion(outputs, labels)

            running_loss += loss.item()
            _, preds = torch.max(outputs, 1)
            correct += torch.sum(preds == labels)

    epoch_loss = running_loss / len(loader)
    epoch_acc = correct.double() / len(loader.dataset)

    return epoch_loss, epoch_acc.item()


best_acc = 0

for epoch in range(EPOCHS):
    print(f"\nEpoch [{epoch+1}/{EPOCHS}]")

    train_loss, train_acc = train_one_epoch(model, train_loader)
    val_loss, val_acc = validate(model, val_loader)

    scheduler.step()

    print(f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f}")
```

```
    print(f"Val   Loss: {val_loss:.4f} | Val   Acc: {val_acc:.4f}")


    if val_acc > best_acc:
        best_acc = val_acc
        torch.save(model.state_dict(), "best_model.pth")
        print("✅ Best model saved!")

print("🎯 Training Complete!")
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[1], line 2
      1 import os
----> 2 import torch
      3 import torch.nn as nn
      4 import torch.optim as optim

ModuleNotFoundError: No module named 'torch'
```

In [ ]: